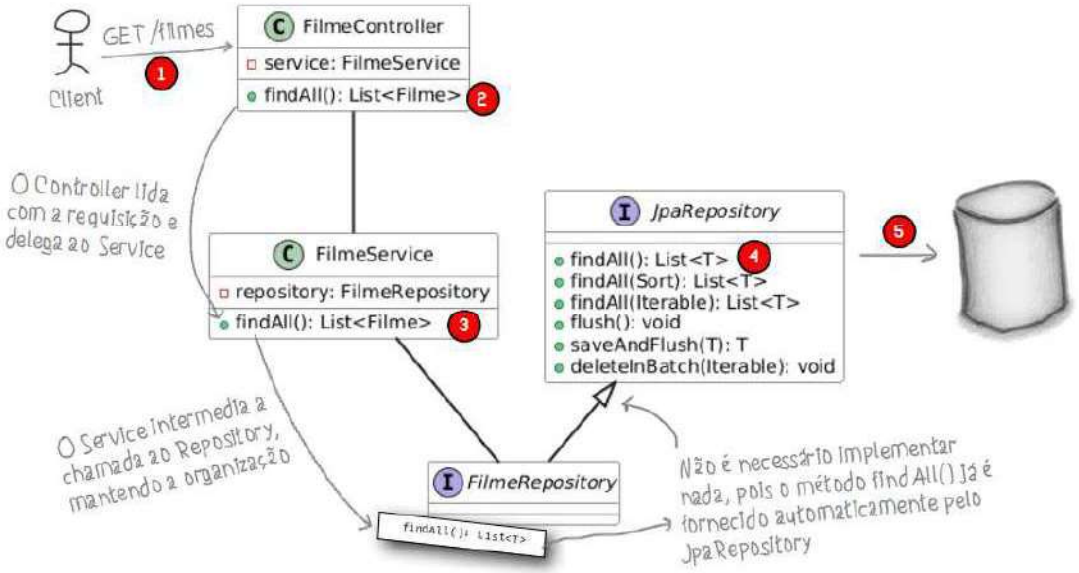
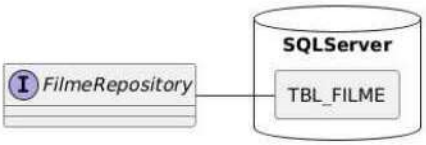


Dessa forma, garantimos a separação de responsabilidades:

- o Controller lida com a requisição,
- o Service com a lógica,
- e o Repository com o acesso aos dados.

2.1. CAMADAS NO SPRING: REPOSITORY

O Repository é responsável por acessar o banco de dados, permitindo realizar operações de CRUD (criar, ler, atualizar, deletar) e consultas complexas em bancos de dados com pouquíssimo código.



O Spring Data JPA elimina a necessidade de escrever classes DAO (Data Access Object) repetitivas, estendendo interfaces básicas (CrudRepository, PagingAndSortingRepository) para gerenciar entidades JPA com o mínimo de código, já que muitas operações já vêm prontas:

```
public interface FilmeRepository extends JpaRepository<Filme, Long> {
}
```

Identificador único (ID) da entidade

Classe que será persistida no banco de dados

Centro Estadual de Educação Tecnológica Paula Souza - CEETEPS
© copyright ETEC Profº Horácio Augusto da Silveira

2.1.1 EXERCÍCIOS: FILMEREPOSITORY

1) Crie o pacote `br.com.etechoracio.ingresso.repository`.

2) Dentro do pacote `repository`, defina uma nova *interface* chamada `FilmeRepository`:

```
public interface FilmeRepository {
}
```

3) Agora faça com que a classe herde de `JpaRepository`.

```
public interface FilmeRepository extends JpaRepository<Filme, Long> {
}
```

2.2. CAMADAS NO SPRING: SERVICE

A camada Service é responsável por concentrar a lógica de negócio da aplicação, atuando como um intermediário inteligente entre Controller e Repository.

Ela é responsável por:

- Aplicar regras de negócio
- Validar dados mais complexos
- Orquestrar chamadas (vários repositories, APIs, etc.)
- Garantir consistência

```
@Service
public class FilmeService {
}
```

A classe Service é anotada com `@Service` indicando ao Spring que ela é um componente gerenciado e pode ser injetada com `@Autowired`.

Por quê não colocar no Controller? O Controller deve apenas receber e responder requisições, não tomar decisões complexas.

2.2.1 EXERCÍCIOS: FILMESERVICE

1) Crie o pacote `br.com.etechoracio.ingresso.service`.

Centro Estadual de Educação Tecnológica Paula Souza - CEETEPS
© copyright ETEC Profº Horácio Augusto da Silveira

2) Dentro do pacote `service`, defina uma nova *classe* chamada `FilmeService` e anote-a com `@Service`.

```
@Service
public class FilmeService {
}
```

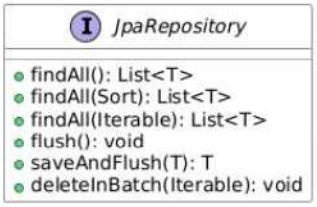
3) Listaremos todos os filmes cadastrados na base de dados usando um objeto da classe `FilmeRepository` injetado com `@Autowired`:

```
@Service
public class FilmeService {

    @Autowired
    private FilmeRepository filmeRepository;

}
```

4) Na hierarquia de `JpaRepository` que `FilmeRepository` herda, temos o método `findAll()`. Observe:

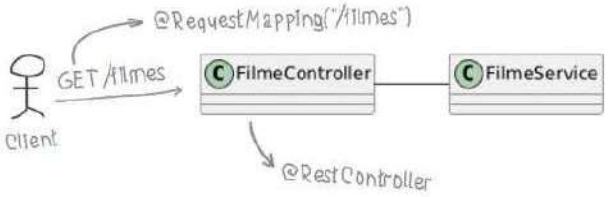


Ou seja, ao usar `JpaRepository`, já temos (quase) tudo, bastando criar um método em `FilmeService` para usar o método da *superclass*:

```
public List<Filme> findAll() {
    return filmeRepository.findAll();
}
```

2.3. CAMADAS NO SPRING: CONTROLLER

Como vimos, um controller não acessa o banco diretamente e não contém regras de negócio complexas, definindo os "endereços" da API.



No Spring, usamos `@RestController` que indica que a classe recebe requisições HTTP e as respostas serão automaticamente convertidas para JSON. Já `@RequestMapping("/filmes")` significa que todas as rotas começam com `/filmes`.

Centro Estadual de Educação Tecnológica Paula Souza - CEETEPS
© copyright ETEC Profº Horácio Augusto da Silveira